



JOHNS HOPKINS

WHITING SCHOOL
of ENGINEERING

FPGA Design Using VHDL

Module 6F: Arithmetic Examples

Arithmetic Operators: numeric_std "+" 1

- Consider the "+" operation on **signed** vs. **unsigned** signals with different widths
 - Signal extension is important and is different for signed vs. unsigned

- **unsigned:**

```
signal a : unsigned (4 downto 0);  
signal b : unsigned (3 downto 0);  
signal result : unsigned(4 downto 0);  
...  
a <= "01101";  
b <= "1011"; -- 01011 after zero ext  
result <= a + b; -- result is 11000
```

- Extend shorter signal with '0'
 - Second value is **zero extended** and then added

Arithmetic Operators: numeric_std "+" 2

- Consider the "+" operation on **signed** vs. **unsigned** signals with different widths
 - Signal extension is important and is different for signed vs. unsigned

- **signed:**

```
signal a : signed (4 downto 0);  
signal b : signed (3 downto 0);  
signal result : signed(4 downto 0);  
...  
a <= "01101";  
b <= "1011"; -- 11011 after sign ext  
result <= a + b; -- result is 01000
```

- Extend shorter signal with MSB of shorter signal
 - Second value is **sign extended** and then added

Arithmetic Operators: numeric_std "+" 3

- A few more notes for completeness:
 - What if a was signed and b was unsigned?
 - This will result in compilation error
 - What if result was 4 bits?
 - `result : (un)signed(3 downto 0);`
 - This will result in compilation error
 - How do we know how extension is performed?
 - signed: Extend with MSB
 - unsigned: Extend with '0'
 - Answer: It's in the `resize` function
 - Why do we know all this?
 - Hint: Look at the `resize` function under `numeric_std` package

Arithmetic Operators: numeric_std "+/-" 4

- The "+" and "-" preserve the length of the data path
 - +/- wrap around on overflow
 - Avoid by resizing before operation
- "+/-" allowed for operands with different sizes
 - Result is the size of the largest operand

```
library ieee;
use ieee.numeric_std.all;
use ieee.std_logic_1164.all;

entity addition is
    port (a,b : in signed (7 downto 0);
          z   : out signed (7 downto 0));
end entity addition;

architecture example of addition is
begin
    -- a + b could possibly wrap if the lengths
    -- are preserved
    z <= a + b;
end architecture addition;
```

```
library ieee;
use ieee.numeric_std.all;
use ieee.std_logic_1164.all;

entity addition is
    port (a,b : in signed (7 downto 0);
          z   : out signed (8 downto 0));
end entity addition;

architecture example of addition is
begin
    -- a + b cannot wrap, but the length has
    -- not been preserved
    z <= resize(a,z'length) + resize(b, z'length);
end architecture addition;
```

Arithmetic Operators: numeric_std "+"

- Examples to illustrate signal lengths

```
signal a, b : signed(7 downto 0);
```

```
...
```

```
a <= "10010101";
```

```
b <= "10110011";
```

- First case when c is defined as 8 bits

```
signal c : signed(7 downto 0);
```

```
...
```

```
c <= a + b; -- results in c = 01001000, overflow
```

```
c <= resize(a, 9) + resize(b, 9); -- compilation error
```

- Second case when c is defined as 9 bits

```
signal c : signed(8 downto 0);
```

```
c <= a + b; -- compilation error
```

```
c <= resize(a, 9) + resize(b, 9); -- results in c = 101001000
```

- What happens if signals were defined as unsigned?

- Assume b <= "00110011". Repeat arithmetic

Arithmetic Operators: numeric_std “-”

- Consider the “-” operation on signed vs. unsigned signals with different widths
 - Sign extension is important
- unsigned:
 - “01101” - “1011” = “01101” - “01011” = “00010”
 - “00101” - “1011” = “00101” - “01011” = “11010”
 - Extend shorter signal with ‘0’
 - Second value is **zero extended** and then subtracted
- signed:
 - “01101” - “1011” = “01101” - “11011” = “10010”
 - Extend shorter signal with MSB of shorter signal
 - Second value is **sign extended** and then subtracted
- Always check answers in decimal

Arithmetic Operators: numeric_std “sign, +/ -, abs”

- Sign +, sign -, abs, +, -, and * operators are available for **signed** vectors only
- +, -, and * available for (un)signed vectors
- Operators defined to wrap on overflow
 - Adding 1 to the most positive signed number returns the most negative signed number (“01..1” + ‘1’ => “10...0”)...
 - Adding 1 to the most positive unsigned number returns 0 (“11..1” + ‘1’ => “00..0”)
- Note that taking abs or negating most negative signed number returns the most negative signed number

Integer	2 Comp
7	0111
6	0110
5	0101
4	0100
3	0011
2	0010
1	0001
0	0000
-1	1111
-2	1110
-3	1101
-4	1100
-5	1011
-6	1010
-7	1001
-8	1000

Arithmetic Operators: numeric_std "*"

- Multiplication "*" does not preserve the data bus size
 - Result size determined by adding the sizes of the operands
- Multiplication does not overflow,
 - Resizing not necessary
 - However, result after multiplication may need to be truncated

```
library ieee;
use ieee.numeric_std.all;
use ieee.std_logic_1164.all;

entity mult is
    port (a,b : in signed (7 downto 0);
          z : out signed (7 downto 0));
end entity mult;

architecture example of mult is
    signal product : signed(15 downto 0);
begin
    --multiply, and then resize
    product <= a*b;
    z <= resize(product, z'length);

    --shorter method
    z <= resize(a*b, 8);
end architecture example;
```

Arithmetic Operators: numeric_std

- We can go through all the operators in the `numeric_std` package ...
 - But we won't
- It's interesting to see the functions for the different operators:
 - http://www.vhdl.org/rassp/vhdl/models/standards/numeric_std.vhd
- However, let's focus a little bit on the compare operator "<"

Comparison Operators in numeric_std

- What happens when we compare unsigned signals of different sizes
- Example:

```
library ieee;  
use ieee.numeric_std_all;  
use ieee.std_logic_1164.all;
```



```
a = "00000000" z = "1111" : altz = '1', agtz = '0'  
a = "10111111" z = "1111" : altz = '0', agtz = '1'  
a = "00011111" z = "1111" : altz = '0', agtz = '1'
```

```
entity test is  
  port (a : in unsigned (7 downto 0);  
        z : in unsigned (3 downto 0);  
        altz, agtz : out std_logic);  
end entity test;
```

```
architecture example of test is  
begin  
  altz <= '1' when a < z else '0';  
  agtz <= '1' when a > z else '0';  
end architecture example;
```

- Numerical Values are compared regardless of size
- What happens if signals are signed instead of unsigned?
- What happens if signals are std_logic_vector?**
- Quiz/Exam question